

Legenda – Concetti di base

- **Derivazione:** Sequenza di applicazioni delle regole di produzione di una grammatica che, a partire dal simbolo iniziale, genera una parola (solo terminali). Può essere sinistra (si riscrive sempre la prima variabile) o destra.
- **Parsing:** Processo inverso alla derivazione: a partire da una parola, si cerca di ricostruire la sequenza di produzioni che la generano, costruendo l'albero di derivazione.
- **Linguaggio:** Insieme di stringhe su un certo alfabeto che soddisfano una certa proprietà, ad esempio tutte le stringhe ben formate secondo una grammatica.
- **Lemma:** Proposizione dimostrata e usata come strumento all'interno di una dimostrazione più ampia (es. il pumping lemma serve a dimostrare che un linguaggio non è regolare).
- **Vocabolario (o alfabeto):** Insieme finito di simboli usati per costruire le parole (es. $\Sigma = \{a, b\}$). Un linguaggio è un sottoinsieme di Σ^* (l'insieme di tutte le stringhe possibili).
- **Produzione:** Regola che descrive come si può riscrivere una variabile in altri simboli (es. $S \rightarrow aSb$).
- **Token:** Unità lessicale riconosciuta dallo scanner, come una parola chiave, un identificatore o un numero.
- **Lessema:** Sequenza di caratteri nel testo che corrisponde a un'istanza concreta di un token (es. `while` è un lessema del token PAROLA_CHIAVE).
- **Albero di derivazione:** Albero che rappresenta graficamente la derivazione di una parola a partire dal simbolo iniziale della grammatica.

1. Espressioni regolari

"Le espressioni regolari sono uno strumento fondamentale per descrivere i linguaggi regolari, cioè quelli accettati da automi a stati finiti. Ogni espressione regolare definisce un linguaggio costruito tramite tre operazioni: unione, concatenazione e chiusura di Kleene. Ad esempio, l'espressione `(a|b)*abb` rappresenta tutte le stringhe su $\{a, b\}$ che terminano in 'abb'." ➡ Esempio: `abb`, `aabb`, `bbabb` sono accettate da questa espressione.

2. Parsing predittivo (in ampiezza, profondità) e deterministico

"Il parsing predittivo è una tecnica top-down che costruisce la derivazione a partire dal simbolo iniziale. Esistono due strategie: in ampiezza (analizza più rami in parallelo) e in profondità (analizza una scelta alla volta, con possibilità di backtracking). Il parsing predittivo deterministico evita il backtracking usando il lookahead per decidere direttamente quale produzione usare." ➡ Esempio: per la

grammatica $S \rightarrow aA \mid bB$, se l'input è aab , il parser predittivo deterministico sceglierà aA grazie al lookahead a .

3. Modello astratto: automi a pila

"Gli automi a pila sono un modello computazionale che estende gli automi a stati finiti con una memoria ausiliaria: la pila. Questo permette di riconoscere linguaggi context-free, come $a^n b^n$. La pila viene usata per ricordare simboli e fare confronti strutturali, come nel caso di parentesi bilanciate."

➡ Esempio: il linguaggio $L = \{ a^n b^n \mid n \geq 0 \}$ può essere riconosciuto da un PDA che inserisce a nella pila e rimuove un a per ogni b letto.

4. Grammatica a struttura di frase

"Una grammatica a struttura di frase è una grammatica context-sensitive, in cui ogni produzione può dipendere dal contesto circostante. Il concetto di derivazione rappresenta l'applicazione successiva delle regole di produzione, mentre una conseguenza è una forma sentenziale ottenuta dopo zero o più applicazioni." ➡ Esempio: con la produzione $A B \rightarrow C D$, la trasformazione avviene solo se A è seguito da B .

5. Come si definisce il linguaggio generato da una grammatica

"Il linguaggio generato da una grammatica G è l'insieme di tutte le parole costituite solo da simboli terminali che possono essere derivate dal simbolo iniziale tramite una sequenza finita di produzioni. Formalmente: $L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$." ➡ Esempio: Se $S \rightarrow aS \mid b$, allora ab , aab , $aaab$, ... appartengono a $L(G)$.

6. Automa a stati finiti non deterministico vs deterministico

"La differenza principale è che un automa deterministico (DFA) ha una e una sola transizione per ogni coppia stato/simbolo, mentre un automa non deterministico (NFA) può avere più transizioni o anche transizioni ϵ . Tuttavia, i DFA e gli NFA riconoscono la stessa classe di linguaggi: i linguaggi regolari." ➡ Esempio: per il linguaggio $(a|b)^*abb$, l'NFA può 'indovinare' quando inizia la parte finale abb , mentre il DFA deve codificare tutto lo stato precedente.

7. Lemma di iterazione per i linguaggi regolari

"Il pumping lemma afferma che ogni parola sufficientemente lunga in un linguaggio regolare può essere 'pompa', cioè ripetuta in una sua sottostringa, senza uscire dal linguaggio. Serve a dimostrare che un linguaggio **non** è regolare per assurdo." ➡ Esempio: $L = \{ a^n b^n \mid n \geq 0 \}$ non è regolare. Se pompriamo la parte con le a , otterremo più a che b , violando la struttura.

8. Gerarchia di Chomsky

"La gerarchia di Chomsky classifica i linguaggi formali in 4 classi: tipo 3 (regolari), tipo 2 (context-free), tipo 1 (context-sensitive), tipo 0 (ricorsivamente enumerabili). Ogni tipo ha grammatiche e macchine equivalenti. Ad esempio, i linguaggi tipo 2 sono generati da grammatiche context-free e riconosciuti da automi a pila." ➡ Esempio: $a^n b^n$ è di tipo 2 ma non di tipo 3. $a^n b^n c^n$ è di tipo 1 ma non di tipo 2.

9. Grammatiche regolari (tipo 3)

"Sono grammatiche in cui tutte le produzioni sono della forma $A \rightarrow aB$ o $A \rightarrow a$ (destra-lineari). Generano esattamente i linguaggi regolari. Sono utili ad esempio nell'analisi lessicale." ➡ Esempio: $S \rightarrow aS \mid b$ genera a^*b , cioè tutte le stringhe di a seguite da un b .

10. Algoritmo CYK

"L'algoritmo CYK verifica se una parola appartiene al linguaggio generato da una grammatica context-free in forma normale di Chomsky. Costruisce una tabella triangolare dove ogni cella contiene le variabili che possono derivare il sottostringa corrispondente. Se alla fine S è nella cella in alto a sinistra, la parola è accettata." ➡ Esempio: Con $w = aabb$, la tabella CYK verifica se esiste una derivazione a partire da S che genera $aabb$.

11. Grammatiche ambigue e non ambigue

"Una grammatica è ambigua se esistono parole che hanno più di un albero di derivazione (cioè più derivazioni sinistre). Questo rende il parsing più difficile o non deterministico. Una grammatica non ambigua genera ogni parola con un solo albero." ➡ Esempio: $E \rightarrow E + E \mid E * E \mid (E) \mid id$ è ambigua perché $id + id * id$ ha due alberi diversi.

12. Derivazione sinistra

"La derivazione sinistra è un tipo di derivazione in cui, ad ogni passo, si riscrive la variabile più a sinistra della forma sentenziale. Questo permette di costruire l'albero di derivazione dal nodo radice alle foglie in modo univoco." ➡ Esempio: $S \rightarrow aS \mid b$, la derivazione sinistra di aab è $S \Rightarrow aS \Rightarrow aaS \Rightarrow aab$.

13. Come passare da grammatica regolare ad automa a stati finiti

"Ogni grammatica regolare può essere trasformata in un automa a stati finiti costruendo uno stato per ogni variabile e una transizione per ogni produzione. Le produzioni $A \rightarrow aB$ diventano transizioni da A a B etichettate da a . Le produzioni $A \rightarrow a$ portano allo stato finale." ➡ Esempio: $S \rightarrow aA, A \rightarrow b$ genera l'automa con transizioni $S \xrightarrow{a} A, A \xrightarrow{b} [finale]$.

14. Analisi lessicale

"L'analisi lessicale è la prima fase della compilazione. Divide il testo in token, riconoscendo i lessemi attraverso automi a stati finiti. Applica la regola del 'lessema più lungo' e risolve conflitti tra token con priorità prefissate." ➡ Esempio: nel testo `int valore = 5;`, lo scanner riconosce `int`, `valore`, `=`, `5`, `;` come token.

15. Automa a pila

"Un automa a pila (PDA) è un automa a stati finiti con una pila, in grado di riconoscere linguaggi context-free. Usa la pila per memorizzare simboli e fare confronti, come nel linguaggio delle parentesi bilanciate o in `a^n b^n`." ➡ Esempio: legge `a` e empila `A`, legge `b` e desapila `A`. Se pila vuota e input finito → accettazione.

16. Dove stanno i linguaggi degli automi a pila nella gerarchia di Chomsky

"Gli automi a pila riconoscono esattamente i linguaggi context-free, quindi corrispondono ai linguaggi di **tipo 2** nella gerarchia di Chomsky." ➡ Esempio: `a^n b^n` è riconosciuto da un PDA ma non da un DFA.

17. Come associamo un grafo ad un automa a stati finiti

"Ogni automa a stati finiti può essere rappresentato come un grafo orientato, in cui:

- i nodi sono gli stati
 - gli archi sono le transizioni etichettate da simboli dell'alfabeto
 - un nodo iniziale è indicato con una freccia in ingresso
 - i nodi finali sono cerchi doppi." ➡ Esempio: un DFA con stati `q0`, `q1`, `q0` iniziale, `q1` finale, e transizione `q0 --a--> q1` si disegna come un grafo.
-

18. Funzione delta estesa

"La funzione delta estesa, denotata con δ^* , descrive come l'automa elabora un'intera stringa. Estende la funzione di transizione δ a stringhe di simboli, applicandola ricorsivamente: $\delta^*(q, \epsilon) = q$, $\delta^*(q, aw) = \delta^*(\delta(q, a), w)$." ➡ Esempio: per DFA con `q0 --a--> q1`, `q1 --b--> q2`, abbiamo $\delta^*(q0, ab) = q2$.

19. Automa minimo

"Un automa minimo è un DFA che riconosce un linguaggio regolare con il numero minimo di stati possibile. Si ottiene unendo stati equivalenti tramite la minimizzazione, che si basa sulla relazione di indistinguibilità." ➡ Esempio: due stati che portano sempre alle stesse accettazioni da quel punto in poi possono essere uniti.

20. Teorema di Nerode

"Il teorema di Nerode caratterizza i linguaggi regolari tramite una relazione di equivalenza: due stringhe u, v sono equivalenti se non esiste un suffisso z tale che solo una tra uz o vz appartiene al linguaggio. Il numero di classi di equivalenza finito implica che il linguaggio è regolare." ➡ Esempio: per $L = (ab)^*$, $u = ab$ e $v = abab$ sono distinguibili perché c'è un z che porta uno dentro il linguaggio e l'altro fuori.